

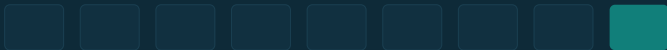


STREAM PROCESSING AVEC APACHE KAFKA

Des pipelines de données à Kafka

Pourquoi le streaming — et comment Kafka a tout changé

ESGI · 5^e année · Data Engineering · 2025–2026



Objectifs de la séance

- **Comprendre la notion de pipeline de données**

Pourquoi le SI moderne doit faire circuler la donnée en quasi temps réel.

- **Situer Kafka dans l'histoire du messaging**

Des files de messages classiques (JMS, RabbitMQ. . .) au log distribué.

- **Saisir l'abstraction du log distribué**

Le modèle mental fondateur de Kafka : découplage, rétention, rejeu.

- **Prendre en main l'environnement**

Lancer un premier cluster (Docker + Kafka UI), produire et consommer.

Au programme

1

Pourquoi des pipelines ?

Le besoin de données en mouvement.

2

Histoire du messaging

Files, pub/sub, ESB et leurs limites.

3

La genèse de Kafka

Le log distribué comme épine dorsale.

4

Atelier + CC

Premier cluster & QCM diagnostique.

01

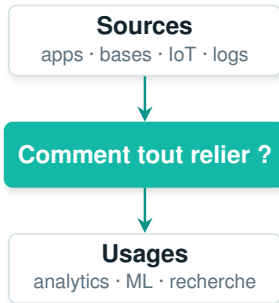
- PARTIE 1

Pourquoi parler de pipelines de données ?

Le SI moderne : des données en mouvement

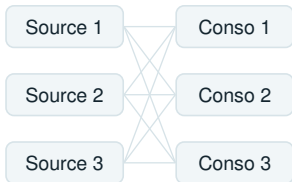
Une organisation produit en permanence des événements — commandes, paiements, clics, logs, mesures de capteurs, changements en base.

- Les sources se multiplient : apps, microservices, bases, IoT, services tiers
- Les consommateurs aussi : analytics, ML, moteur de recherche, alerting
- L'attente : du quasi temps réel — plus seulement du batch nocturne



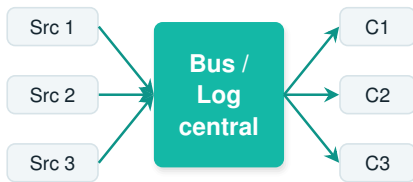
Sans point central : l'explosion des connexions

Point à point



$N \times M$ liaisons fragiles

Avec un point central



$N + M$: chaque système
ne se branche qu'une fois

02

- PARTIE 2

Petite histoire du messaging

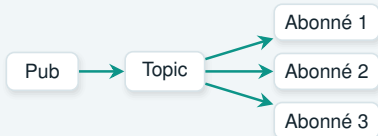
Deux modèles : file vs publication/abonnement

■ File de messages (point-à-point)



- Un message → un seul consommateur
- Message supprimé après lecture
- Idéal pour répartir des tâches

■ Publication / abonnement (topic)



- Un message → tous les abonnés
- Diffusion (broadcast)
- Idéal pour propager des événements

Kafka réconciliera les deux modèles — on y reviendra.

Les brokers de messages « classiques »

ActiveMQ / JMS

standard Java

RabbitMQ

routage AMQP

IBM MQ

entreprise

TIBCO EMS

temps réel

Ce qu'ils apportent

- Files durables & accusés de réception (ack)
- Routage riche (RabbitMQ : exchanges, bindings)
- Transactions, priorités, expiration (TTL)

Pensés pour

- L'intégration applicative & la distribution de tâches
- Des messages transactionnels, fiables
- Des volumes modérés, une conservation courte

ESB et brokers classiques : les limites

Files de messages et bus d'entreprise (ESB) ont structuré l'intégration — mais ils montrent leurs limites face au temps réel et au Big Data.

■ Rétention courte

Message consommé = supprimé. Pas d'historique conservé.

■ Pas de rejou

Impossible de relire les événements passés.

■ Montée en charge difficile

Scalabilité horizontale limitée, débit plafonné.

■ Couplage & centralisation

L'ESB devient un point de contention et de complexité.

03

- PARTIE 3

La genèse de Kafka

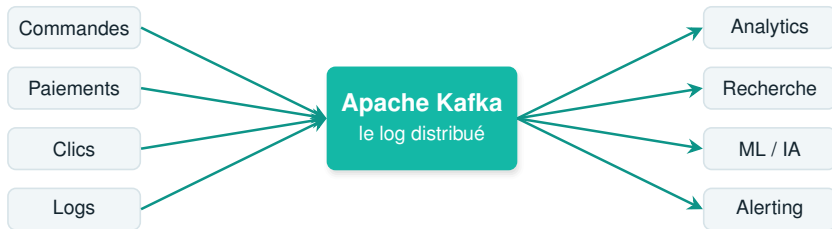
D'un problème d'échelle à une idée simple

Chez LinkedIn, vers 2010 : des dizaines de systèmes (profils, messagerie, recherche, analytics) à connecter entre eux. Le résultat : un enchevêtrement d'intégrations point-à-point ingérable.

- **L'idée : tout faire passer par un journal central unique**
- Open-source en 2011, puis projet Apache
- Le nom « Kafka » : un clin d'œil de l'équipe (J. Kreps) à l'écrivain

« Et si la donnée n'était plus poussée de système en système, mais publiée une fois dans un log que chacun lit à son rythme ? »

Le log distribué comme épine dorsale



Chaque source publie une fois ; chaque usage lit ce dont il a besoin, indépendamment.

Qu'est-ce qu'un log ?

- Ajout uniquement (*append-only*) : on n'écrit qu'à la fin
- Ordonné : chaque enregistrement reçoit un numéro, l'*offset*
- Immuable : on ne modifie ni ne supprime — on relit
- Persisté sur disque, séquentiellement (donc très rapide)

Un topic = un log : une suite ordonnée d'événements



L'offset (0, 1, 2...) identifie la position de chaque événement dans le log.

Producteurs & consommateurs sur le log



- Plusieurs consommateurs lisent le même log, indépendamment
- Chacun mémorise son offset — sa propre position de lecture
- On peut rejouer : revenir à un offset passé et tout relire

Pourquoi le log change tout

■ Découplage

Producteurs et consommateurs ne se connaissent pas.

■ Rétention

Les événements restent : minutes, jours, ou toujours.

■ Rejeu

Relire le passé : reprise, debug, nouveaux usages.

■ Multi-consommateurs

Un même flux alimente analytics, ML, alerting...

■ Débit élevé

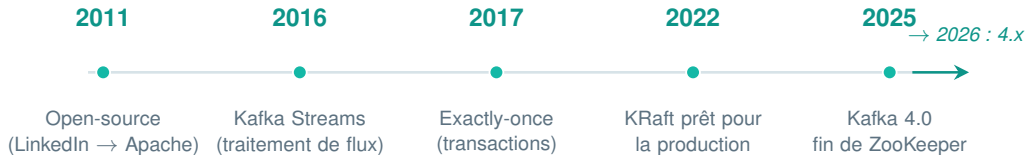
Écriture séquentielle + partitions = scalable.

■ Ordre garanti

L'ordre des événements est préservé (par partition).

L'ÉVOLUTION

De LinkedIn à la plateforme d'aujourd'hui



L'écosystème Kafka en bref

Apache Kafka 4.x — le log distribué (KRaft, sans ZooKeeper)

■ Clients

Producteurs & consommateurs :
APIs pour publier et lire.

■ Kafka Connect

Brancher bases, fichiers, services — sans code.

■ Kafka Streams

Transformer, agréger, joindre en temps réel (Java).

■ Schema Registry

Contrats de données : Avro/Protobuf & compatibilité.

On explorera chacune de ces briques au fil des séances.

Kafka vs file de messages classique

Critère	File classique	Apache Kafka
Modèle de consommation	1 message → 1 consommateur	1 flux → N consommateurs indépendants
Après lecture	message supprimé	conservé (rétention configurable)
Rejeu du passé	non	oui — revenir à un offset
Débit & scalabilité	modéré	très élevé (partitions)
Ordre	par file	par partition

Kafka n'est pas toujours la réponse : pour de la simple distribution de tâches, une file reste pertinente — et Kafka 4.x sait aussi le faire avec les share groups.

Cas d'usage typiques

■ Microservices

Communication asynchrone entre services.

■ Pipelines / ETL

Acheminer et transformer la donnée en continu.

■ Stream analytics

Tableaux de bord et métriques en direct.

■ Logs & métriques

Centraliser télémétrie et journaux.

■ Change Data Capture

Capter les changements des bases de données.

■ Event sourcing

Le log comme source de vérité des états.

04

• PARTIE 4 · ATELIER

Votre premier cluster Kafka

Docker Compose + Kafka UI

- Un cluster local jetable, lancé en une seule commande
- Un broker en mode KRaft (Kafka 4.x, sans ZooKeeper)
- Kafka UI pour visualiser topics, partitions et messages
- Prérequis : Docker — et Java 17+ pour la suite du cours

`docker-compose.yml`

```
services:
  broker:
    image: apache/kafka:4.3.0
    ports: ["9092:9092"]
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES:
        broker,controller
  kafka-ui:
    image: provectuslabs/kafka-ui
    ports: ["8080:8080"]
```

Produire et consommer vos premiers messages

- 1 Lancer le cluster (up -d)
- 2 Ouvrir Kafka UI (:8080)
- 3 Créer le topic premier-topic
- 4 Produire des messages en console
- 5 Consommer (--from-beginning)
- 6 Observer : partitions & offsets

bash

```
# creer un topic
kafka-topics.sh --create \
  --topic premier-topic \
  --bootstrap-server localhost:9092

# produire
kafka-console-producer.sh \
  --topic premier-topic \
  --bootstrap-server localhost:9092

# consommer depuis le debut
kafka-console-consumer.sh \
  --topic premier-topic \
  --from-beginning \
  --bootstrap-server localhost:9092
```

QCM diagnostique d'entrée

- **Objectif : calibrer le cours — pas vous piéger**
- Couvre : Kafka, Java, Spark, Flink et la gestion de code (Git)
- $\approx$ 27 questions · 30–35 min · auto-corrigé (Google Form)
- Un score faible sur une partie = on la renforce ensemble

Au menu du QCM

- ✓ Apache Kafka
- ✓ Java
- ✓ Apache Spark
- ✓ Apache Flink
- ✓ Git & gestion de code

Récapitulatif

- ✓ Un pipeline fait circuler la donnée en continu, en quasi temps réel.
- ✓ Le messaging a évolué : files → publication/abonnement → log distribué.
- ✓ Kafka, c'est un log : ordonné, immuable, persistant, lu via des offsets.
- ✓ Le log apporte découplage, rétention, rejeu et multi-consommateurs.
- ✓ Vous avez désormais un cluster qui tourne, et qui produit/consomme.



LA SUITE — SÉANCE 2

Anatomie distribuée de Kafka

Topics, partitions, offsets, brokers et le mode KRaft — puis un TP sur un cluster à 3 brokers pour manipuler partitions et réplication.